

L10c: Stochastic Multi-Armed Bandits

Exploration, regret, UCB1, and Thompson sampling

CHEME 5800 Cornell University

Copyright Jeffrey Varner 2026. All Rights Reserved

Objectives for Today

How should an agent choose among actions with unknown rewards, when it sees only the reward of the action it takes?

Today's concepts

- Balance exploration and exploitation: trying actions to learn about them versus taking the action that looks best so far.
- Solve multi-armed bandit problems: choose the next arm with explore-first, epsilon-greedy, UCB1, and Thompson sampling.
- Compare bandit algorithms with regret: define regret and compare the bounds of explore-first, epsilon-greedy, and UCB1.

Lecture: Stochastic Multi-Armed Bandit Problems

Lecture and Example

Lecture: Stochastic Multi-Armed Bandit Problems
In this lecture, we define the bandit problem and its regret, and work through four algorithms for choosing the next arm.

Example: Step Through the UCB1 Algorithm One Round at a Time
In this example, three sets of reaction conditions are the arms of a bandit. We play the first rounds of UCB1 by hand and predict each pull before we compute it.

Exploration vs. Exploitation

An agent chooses an action, receives a reward, and uses it to decide what to try next, aiming to collect as much reward as possible.

- Exploration: try less-tried actions to learn their rewards.
- Exploitation: take the action that looks best so far. An agent that only exploits never tries anything new; how boring!

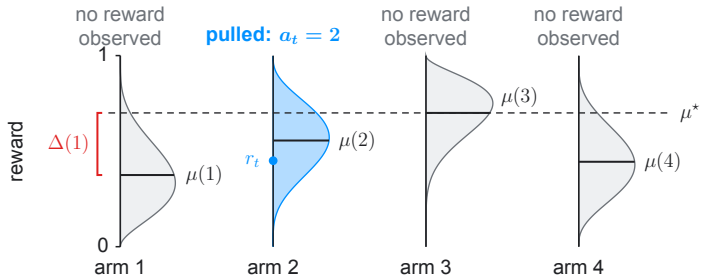
Explore too much, and we give up reward now. Exploit too much, and we may write off an action that was just unlucky.

Discuss

Think of a choice you make again and again, such as where to eat lunch. How do you decide when to try something new and when to go back to your favorite?

What Is a Bandit Problem?

An agent repeatedly chooses among K options, called arms, and receives a reward from the arm it pulls.



hidden from the agent: the shapes, the means, $\mu^\star$, and $\Delta(1)$

The agent sees only the blue dot. In round t, it pulls arm a_t and sees one reward r_t. It does not know $\mu^\star$ or which arm has it, but enough pulls will reveal both. The question is how to choose the pulls.

Bandit Problems in Practice

Each system tries one option at a time and sees only its outcome:

Application	Arms	Reward of one pull	Cost of exploring
Clinical trials Villar et al., 2015	treatments	1 if it works, 0 if not	some patients get a worse treatment
News recommendation Li et al., 2010	front-page articles	1 if clicked, 0 if not	some visitors see a less popular article
Network routing Talebi et al., 2018	network paths	data arrival time	some data take a slower path

Bandit Problems in Practice (continued)

Every exploratory pull has a real cost:

Application	Arms	Reward of one pull	Cost of exploring
Online pricing Misra et al., 2019	candidate prices	profit, or 0 if no sale	some customers see a worse price
Reaction conditions Wang et al., 2024	catalyst, solvent, and base	yield, from 0 to 1	some runs use poor conditions

Arrival time and profit do not lie between 0 and 1, so we rescale them to $[0, 1]$, with a faster arrival or a higher profit closer to 1.

Stochastic Bandit Model

The agent has a fixed set of arms $\mathcal{A}$, with $|\mathcal{A}| = K$, and plays T rounds. Arm a draws rewards from $\mathcal{D}_a$ on $[0, 1]$, with mean reward $\mu(a) = \mathbb{E}_{r \sim \mathcal{D}_a}[r]$.

For $t = 1, 2, \ldots, T$ **do**:

1. The agent chooses an arm a_t from its past actions and rewards.
2. The environment independently draws $r_t \sim \mathcal{D}_{a_t}$.
3. The agent receives r_t and updates its estimates.

The bounds assume independent draws, stationary distributions that do not change, and rewards in $[0, 1]$. Maximizing total reward means minimizing regret.

Regret

Regret is the reward an agent loses while it learns:

$$R(T) = \underbrace{T\mu^\star}_{\text{best-arm}} - \underbrace{\sum_{t=1}^{T} r_t}_{\text{our reward}}$$

Each pull of arm a earns, on average, the gap $\Delta(a) = \mu^\star - \mu(a)$ less than a pull of the best arm. Adding the gaps of the pulled arms gives the gap loss:

$$G(T) = \sum_{t=1}^{T} \underbrace{\Delta(a_t)}_{\text{lost pull } t}$$

The agent cannot compute $R(T)$ or $G(T)$, because they use the arm means; we compute them when we make up the arms to test an algorithm. Rewards are random, so $R(T)$ and $G(T)$ vary from game to game; on average, $\boxed{\mathbb{E}[R(T)] = \mathbb{E}[G(T)]}$.

Discuss

Two algorithms choose among the same arms, and by the last round both are pulling the best arm. Did they lose the same amount of reward? Why or why not?

Explore-First Algorithm

Pull each arm N times (exploration), then pull the arm with the highest sample mean (exploitation).

Initialize: Given K arms and $T \geq K$ rounds, choose

$$N \approx \left(\frac{T}{K}\right)^{2/3} (\log T)^{1/3},$$

rounded to the nearest integer, with at least 1 and at most $\lfloor T/K \rfloor$ pulls per arm.

1. For each arm $a \in \mathcal{A}$ **do**: pull arm a for N rounds and compute its sample mean $\bar{\mu}(a)$.
2. Choose $\hat{a} \leftarrow \arg\max_{a \in \mathcal{A}} \bar{\mu}(a)$; ties go to the first arm.
3. For $t = KN + 1, \ldots, T$ **do**: pull arm $\hat{a}$.

Explore-First: Regret Bound

With this choice of N, explore-first achieves (Slivkins, Theorem 1.5):

$$\boxed{\mathbb{E}[R(T)] \leq \mathcal{O}\left(T^{2/3}(K \log T)^{1/3}\right)}$$

- Too few exploration pulls, and we may commit to the wrong arm.
- Too many, and we waste pulls on poor arms.

This N balances the two costs. It explores for a fraction $KN/T \approx (K \log T/T)^{1/3}$ of the game, which shrinks as T grows.

Epsilon-Greedy Algorithm

Pull a random arm with a shrinking probability ϵ_t, and otherwise the best arm so far.

Initialize: Given K arms and $T \geq K$ rounds, pull each arm once; set $N_a \leftarrow 1$ and $\bar{\mu}(a) \leftarrow r_a$, its first reward.

For $t = K+1, \ldots, T$ **do**:

1. Compute the exploration probability $\epsilon_t = \min\left\{1,\ \underbrace{(K \log t/t)^{1/3}}_{>1 \text{ in early rounds}}\right\}$.
2. Choose the arm a_t: draw $u \sim \text{Uniform}(0,1)$.
   - If $u \leq \epsilon_t$ **then** choose a_t at random (exploration).
   - Otherwise, $a_t \leftarrow \arg\max_{a \in \mathcal{A}} \bar{\mu}(a)$; ties go to the first arm (exploitation).
3. Pull arm a_t and receive r_t.
4. Update N_{a_t} and $\bar{\mu}(a_t)$.

This ϵ_t is the explore-first exploration fraction with T replaced by t, spread over the game.

Optimism Under Uncertainty

Assume each arm is as good as the data so far allow, and pull the arm with the best optimistic estimate.

The upper confidence bound of arm a at round t is $\bar{\mu}(a) + U(a, t)$. The confidence radius is an uncertainty bonus that is large when arm a has few pulls:

$$U(a, t) = \sqrt{\frac{2 \log t}{N_a}},$$

where N_a counts the pulls of arm a before round t.

With high probability, the bound lies above the true mean $\mu(a)$. This leads to UCB1 (Auer et al., 2002).

UCB1 Algorithm

Initialize: Given K arms and $T \geq K$ rounds, pull each arm once; set $N_a \leftarrow 1$ and $\bar{\mu}(a) \leftarrow r_a$, its first reward.

For $t = K+1, \ldots, T$ **do**:

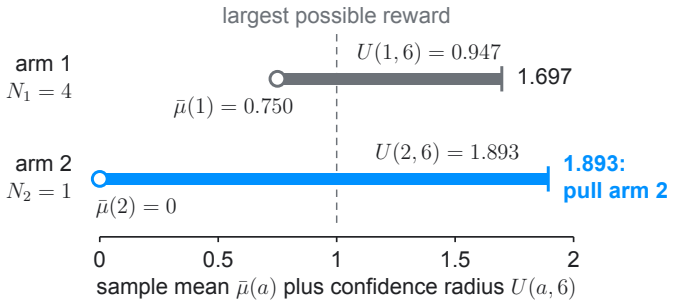
1. Compute the confidence radius $U(a,t)$ for each arm $a \in \mathcal{A}$.
2. Choose the arm with the largest upper confidence bound, $a_t \leftarrow \arg\max_{a \in \mathcal{A}} \{\bar{\mu}(a) + U(a,t)\}$; ties go to the first arm.
3. Pull arm a_t and receive a reward r_t.
4. Update N_{a_t} and $\bar{\mu}(a_t)$.

UCB1 achieves (Slivkins, Theorem 1.15):

$$\boxed{\mathbb{E}[R(T)] \leq \mathcal{O}\left(\sqrt{KT \log T}\right)}$$

Worked Example: Two Arms, Five Rounds

After five rounds, arm 1 has four pulls with $\bar{\mu}(1) = 0.75$, and arm 2 has one pull with $\bar{\mu}(2) = 0$. Before round $t = 6$:



One observation leaves more uncertainty than four. UCB1 pulls arm 2: its bound, 1.893, beats arm 1's 1.697. A bound can exceed 1, the largest possible reward, because the bonus is large when an arm has few pulls.

Example: Step Through UCB1

In this example, three sets of reaction conditions are the arms; an experiment succeeds if the yield clears a target.

1. Build the arms, and see how a few experiments can rank them in the wrong order.
2. Play the first rounds of UCB1 by hand: predict each pull, then compute $\bar{\mu}(a)$, $U(a, t)$, and the bound of every arm.
3. Play a full game, check it against the course package, and find which arms cost the most reward.

Example: Step through the UCB1 algorithm one round at a time

Comparing the Regret Bounds

Algorithm	How it explores	Expected-regret bound
Explore-first	pulls every arm N times, then commits	$\mathcal{O}\left(T^{2/3}(K\log T)^{1/3}\right)$
Epsilon-greedy	pulls a random arm with a shrinking probability ϵ_t	$\mathcal{O}\left(T^{2/3}(K\log T)^{1/3}\right)$
UCB1	pulls the arm with the largest upper confidence bound	$\mathcal{O}\left(\sqrt{KT\log T}\right)$

All three bounds grow more slowly than T, so the regret per round shrinks toward zero. The UCB1 bound grows roughly like $\sqrt{T}$, more slowly than $T^{2/3}$, so UCB1 has the strongest guarantee of the three.

Bounds: Slivkins, Theorems 1.5, 1.6, and 1.15.

Discuss

UCB1 has the strongest guarantee, yet explore-first is still common in practice. When might it be the better choice?

Binary Bernoulli Bandit Problem

A special case: each reward is binary, $r_t \in \{0, 1\}$. Arm a pays 1 with an unknown probability θ_a, so $r \sim \texttt{Bern}(\theta_a)$:

$$\texttt{Bern}(r; \theta) = \begin{cases} \theta & \text{if } r = 1 \\ 1 - \theta & \text{if } r = 0 \end{cases}, \qquad \mathbb{E}[r] = \theta, \quad \text{Var}[r] = \theta(1 - \theta).$$

- Reward model: the agent knows the form of each reward distribution, but not its success probability.
- Binary: an action can be a complex experiment, as long as its outcome is true or false.

So $\mu(a) = \theta_a$, and the agent must learn which arm has the largest θ_a.

From Zeros and Ones to a Probability

The reward of one pull is an indicator, $r = \mathbb{1}\{\text{success}\}$: 1 if the action succeeds, 0 if it fails.

- With this coding, rather than the ± 1 we used for class labels, the sum of the rewards counts the successes.
- After n pulls of arm a, the sample mean is the fraction of successes, and it approaches θ_a as n grows:

$$\bar{\mu}(a) = \frac{1}{n}\sum_{i=1}^{n} r_i = \frac{\text{number of successes}}{n} \longrightarrow \theta_a$$

A Beta Model of Each Arm

Ready to get your mind blown? Instead of one estimate of each θ_a, we keep a distribution that describes our uncertainty about it:

- The prior is our belief before any rewards. $\mathrm{Beta}(\alpha, \beta)$ describes an unknown probability on $[0, 1]$; its mean is $\alpha/(\alpha + \beta)$, and $\mathrm{Beta}(1, 1)$ is uniform.
- After S successes and F failures, the posterior, our updated belief, is again a Beta distribution:

$$\boxed{\mathrm{Beta}(\alpha, \beta) \;\longrightarrow\; \mathrm{Beta}(\alpha + S,\, \beta + F)}$$

Epsilon-Greedy for the Bernoulli Bandit

The epsilon-greedy algorithm carries over with two changes:

1. Track the successes $S(a)$ and failures $F(a)$ of each arm, starting from $S(a) = F(a) = 0$ and uniform priors, $\alpha(a) = \beta(a) = 1$.
2. When exploiting, pull the arm with the highest posterior mean:

$$a_t \leftarrow \arg\max_{a \in \mathcal{A}} \left\{ \frac{\alpha(a) + S(a)}{\alpha(a) + \beta(a) + S(a) + F(a)} \right\}.$$

After each pull, update the counts of the arm we pulled:

$$S(a_t) \leftarrow S(a_t) + r_t, \qquad F(a_t) \leftarrow F(a_t) + (1 - r_t).$$

Everything else is unchanged: pull each arm once, then explore at random with probability ϵ_t.

Thompson Sampling Algorithm

Initialize: Given K arms and $T \geq 1$ rounds, set $\alpha(a) \leftarrow 1$, $\beta(a) \leftarrow 1$, $S(a) \leftarrow 0$, and $F(a) \leftarrow 0$ for each arm $a \in \mathcal{A}$.

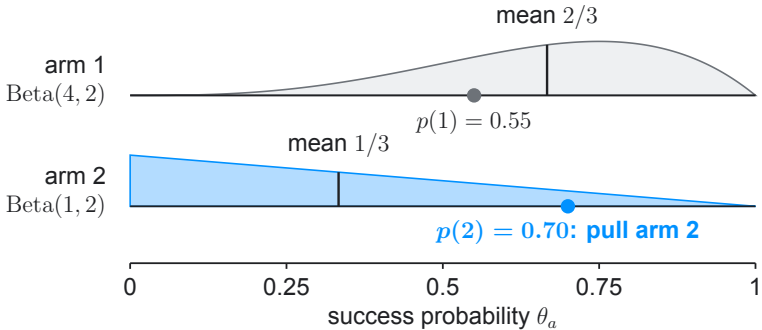
For $t = 1, 2, \ldots, T$ **do**:

1. Sample $p(a) \sim \text{Beta}\left(\alpha(a) + S(a),\ \beta(a) + F(a)\right)$ for every arm $a \in \mathcal{A}$, a sampled guess at its success probability θ_a.
2. Choose the arm with the highest sample, $a_t \leftarrow \arg\max_{a \in \mathcal{A}} \{p(a)\}$.
3. Pull arm a_t and receive $r_t \in \{0, 1\}$.
4. Update $S(a_t) \leftarrow S(a_t) + r_t$ and $F(a_t) \leftarrow F(a_t) + (1 - r_t)$.

It explores mostly arms that could still be the best, so it often beats epsilon-greedy (Russo et al., 2020). Thompson (1933) proposed it for choosing between two treatments.

Worked Example: Updating a Bernoulli Arm

Both arms start from uniform $\mathrm{Beta}(1,1)$ priors. Arm 1 has 3 successes and 1 failure, and arm 2 has 1 failure.



Sampling gives an uncertain arm another trial. Arm 2's sample wins, so it is pulled; a success makes its posterior $\mathrm{Beta}(2,2)$, mean $1/2$.

Discuss

Suppose each arm is a set of reaction conditions and each pull is one experiment, a success if the yield clears a target. After a few weeks of experiments, which arms would you keep testing, and how do the posteriors help you decide?

Lab: Bandit Algorithms

In the lab, we put these algorithms to work:

- we run explore-first, epsilon-greedy, and UCB1 on a bandit with several arms, average their regret over many games, and compare how fast it grows with the bounds,
- we implement Thompson sampling for a binary Bernoulli bandit and watch the posterior of the best arm concentrate around its true success probability, while the arms it rarely pulls stay uncertain.

Lab: Bandit algorithms

Summary

We introduced bandit problems and four ways to balance exploration and exploitation.

Key takeaways

- **Exploration and exploitation.** Too little exploration can lock an agent onto an arm that only looked good by chance.
- **Bandit algorithms.** The four rules explore on a fixed schedule, at random, with an optimism bonus, or by sampling.
- **Regret bounds.** The UCB1 bound grows like $\sqrt{T}$, more slowly than the $T^{2/3}$ bounds of the other two.

Every algorithm learns from bandit feedback: the agent sees the reward of the arm it pulls and nothing about the others.